
TN 15.2.0.0: Adjust 3D Data Scaling and Offset

Release 2025.09

Heliotis AG

Aug 29, 2025

Contents

1	Introduction	1
1.1	Challenge	1
2	Configure Custom Surface Scaling	2
3	Configure Custom Surface Offset	2
4	Calculate Real-World Coordinates Data	2
5	Document History	7

1 Introduction

heliInspect devices scale 3D output data in *um* and map the value to the current stage position which is in many cases a negative position

This application note describes how to setup 3D data scaling and offset. This allows a well configured value range for *unsigned int16* (Mono16) surface output.

The required features described in this documentation are available since software version **1.9.0**.

1.1 Challenge

Some applications could not handle the 3D data output as 32bit floating point value. For such applications, the heliInspect H8/H9 as well as any other heliBrd B4 based camera systems from heliotis supports an 16bit unsigned integer data output. Within this mode, the output values are integers and its range is limited from 0 - 65535.

As a default configuration, the camera scales the 3D output data in *um* and maps the value to the current stage position which is in many cases a negative position. In section *Configure Custom Surface Scaling* the configuration for custom output data scaling is described. Section *Configure Custom Surface Offset* contains the configuration for a custom surface shift along the z direction and finally, the section *Calculate Real-World Coordinates Data* contains the calculations and additional information how the submitted 3D data are transferred back to the *real-world* data.

Note: The Scaling and Offset configuration is applied to any 3D Data output data (16bit integer as well as 32bit floating point).

2 Configure Custom Surface Scaling

A rough scaling setting is applied with the feature *Scan3dDistanceUnit* which allows to select the data output unit as *mm*, *um* or *nm*. The default value, and in almost all cases the best selection is *um*.

To adjust a custom scaling the feature *Scan3dCoordinateScale[Scan3dCoordinateSelector]* is available. The reciprocal value of the *CoordinateC* configuration is internally multiplied with the real-world z value. As an example, the following code listening shows the configuration to adjust the z values to 100nm per integer step.

Listing 2.1: Scan3dCoordinateScale example

```
Scan3dDistanceUnit = "um"  
Scan3dCoordinateSelector = "CoordinateC"  
Scan3dCoordinateScale = 0.1
```

3 Configure Custom Surface Offset

The feature *Scan3dCoordinateOffset[Scan3dCoordinateSelector]* allows the shifting of the 3D output data along the z axis. The value configured within the *CoordinateC* selector is internally subtracted from the real-world z values. As an example, the following code listening shows the configuration to shift the the surface 14.1mm upper. This place a real-world pixelvalue from -13900um up to 200um.

Listing 3.1: Scan3dCoordinateOffset example

```
Scan3dDistanceUnit = "um"  
Scan3dCoordinateSelector = "CoordinateC"  
Scan3dCoordinateOffset = -14100.0
```

Note: The unit of the *Scan3dCoordinateOffset* feature is configured with the *Scan3dDistanceUnit* feature. It is recommended to select a well *Scan3dDistanceUnit* before tuning the *Scan3dCoordinateOffset* value.

4 Calculate Real-World Coordinates Data

The calculation of the 3d data to real-world data requires the configuration value of the *Scan3dCoordinateScale[Scan3dCoordinateSelector]* and *Scan3dCoordinateOffset[Scan3dCoordinateSelector]* features. The values are readable from the device configuration by reading the two features directly or optionally included in the data buffers meta data as so-called chunk data.

The first listening [Listing 4.1](#) shows how to read the information from the device configuration and the second listening [Listing 4.2](#) how to enable the chunk data and read the information from the buffer chunk data. For the final calculation one of this two approaches could be selected.

Listing 4.1: read Scan3dCoordinateScale and Scan3dCoordinateOffset example

```
Scan3dCoordinateSelector = "CoordinateC"
scaleC = Scan3dCoordinateScale
offsetC = Scan3dCoordinateOffset
```

Listing 4.2: enable Chunk data and read Scan3dCoordinateScale and Scan3dCoordinateOffset from the data buffer

```
# enable the chunk data
ChunkModeActive = True
ChunkSelector = "Scan3dCoordinateScale"
ChunkEnable = True
ChunkSelector = "Scan3dCoordinateOffset"
ChunkEnable = True

# after an acquisition, the configuration is readable from the buffers chunk data
ChunkScan3dCoordinateSelector = "CoordinateC"
scaleC = ChunkScan3dCoordinateScale
offsetC = ChunkScan3dCoordinateOffset
```

The following pseudo code shows the transformation of the submitted 3d data to real-world coordinate.

Listing 4.3: pseudo code how to transform the 3d Data to real-world coordinate data

```
for(row = 0; row < Height; row++)
{
    for(col = 0; col < Width; col++)
    {
        coordC[row,col] = imageC[row,col] * scaleC + offsetC;
    }
}
```

Listing 4.4: test 42

Listing 4.5: Python example demonstrate this application node

```
1  ##
2  # \file          an_15.2.x.y.py
3  # \author        Silvan Murer
4  # \date          04 Aug 2022
5  # \copyright     Heliotis, 2022
6  #
7  # \brief         Example for an_15.2.x.y
8  import os, sys
9  import time
10 from harvesters.core import Harvester
11 import genicam.genapi
12 import matplotlib.pyplot as plt
13 from mpl_toolkits.axes_grid1 import make_axes_locatable
14 import numpy as np
15 import statistics
16 from collections import OrderedDict, Mapping
17 import json
18
19 import warnings
20 warnings.simplefilter('always') # always, ignore
```

(continues on next page)

```

21
22
23 ##
24 # Scan for devices and let the user select one
25 #
26 # \param h harvesters object
27 # \return harvesters camera object
28 def selectDevice(h):
29     h.update()
30     print(str(len(h.device_info_list)) + " devices detected on the Network\n\n")
31     cnt = 0
32     for dev in h.device_info_list:
33         cnt = cnt + 1
34         msg = str(cnt) + " " + dev.id_ + " (sn:" + dev.serial_number + ")"
35         print(msg)
36
37     selection_str = input("Select a device (0=exit): ")
38     selection_int = int(selection_str)
39     if((selection_int <= 0) or (selection_int > cnt)):
40         exit('No device selected - exit script')
41
42     deviceID = h.device_info_list[selection_int-1].id_
43     print('selected device:', deviceID)
44     return h.create(selection_int-1)
45
46
47 ##
48 # initialize the camera with some default parameters
49 #
50 # \param camera camera handler
51 def initializeDevice(camera):
52     # enable required components
53     camera.remote_device.node_map.ComponentSelector.value = "Intensity"
54     camera.remote_device.node_map.ComponentEnable.value = False
55     camera.remote_device.node_map.ComponentSelector.value = "Range"
56     camera.remote_device.node_map.ComponentEnable.value = True
57     camera.remote_device.node_map.ComponentSelector.value = "Reflectance"
58     camera.remote_device.node_map.ComponentEnable.value = False
59     camera.remote_device.node_map.ComponentSelector.value = "Phase"
60     camera.remote_device.node_map.ComponentEnable.value = False
61
62     # trigger configuration
63     camera.remote_device.node_map.TriggerSelector.value = "RecordingStart"
64     camera.remote_device.node_map.TriggerMode.value = "On"
65     camera.remote_device.node_map.TriggerSource.value = "Stage"
66     camera.remote_device.node_map.TriggerSelector.value = "AcquisitionStart"
67     camera.remote_device.node_map.TriggerMode.value = "Off"
68     camera.remote_device.node_map.TriggerSelector.value = "FrameStart"
69     camera.remote_device.node_map.TriggerMode.value = "On"
70     camera.remote_device.node_map.TriggerSource.value = "Software"
71
72     # motion and position configuration
73     camera.remote_device.node_map.ScanPosition.value = -14.5
74     camera.remote_device.node_map.ScanRange.value = 0.5
75     camera.remote_device.node_map.ScanSpeed.value = 5.0
76     camera.remote_device.node_map.GeneralSpeed.value = 10.0
77
78     camera.remote_device.node_map.ScanMode.value = "Down"
79     camera.remote_device.node_map.StageInit.execute()
80
81     # additional camera and processing configuration

```

```

82     camera.remote_device.node_map.Scan3dExtractionMethod.value =
83     ↪ "AcceleratedCenterOfMassIQCorrection"
84     camera.remote_device.node_map.Scan3dScalingMethod.value = "zTags"
85     camera.remote_device.node_map.Scan3dDistanceUnit.value = "um"
86
87     camera.remote_device.node_map.TargetVerticalSpacing.value = 2.5
88     camera.remote_device.node_map.ExposureRatio.value = 1.0
89
90     camera.remote_device.node_map.FPNCorrection.value = "AverageLastFrames"
91     camera.remote_device.node_map.FPNCorrectionNFrames.value = 8
92     camera.remote_device.node_map.ExtSimpMaxHWin.value = 7
93
94     heliDriverFamily = camera.remote_device.node_map.HeliDriverFamily.value
95     # illumination control (D3)
96     if (heliDriverFamily == "D3"):
97         camera.remote_device.node_map.LightControllerSelector.value =
98         ↪ "LightController0"
99         camera.remote_device.node_map.LightControllerSource.value = "UserOutput0"
100        camera.remote_device.node_map.LightBrightness.value = 100.0
101        # illumination control (D2)
102        elif (heliDriverFamily == "D2"):
103            camera.remote_device.node_map.LineSelector.value = "Line2"
104            camera.remote_device.node_map.LineSource.value = "UserOutput0"
105            camera.remote_device.node_map.LineInverter.value = 1
106            # switch on the illumination (D2/D3)
107            camera.remote_device.node_map.UserOutputSelector.value = "UserOutput0"
108            camera.remote_device.node_map.UserOutputValue.value = True
109
110        print('*****')
111        print('* ' + str(os.path.basename(__file__)))
112        print('*****')
113
114    try:
115        ctiFile = os.environ['DIAPHUS_GENTL64_FILE']
116
117        h = Harvester()
118        h.add_file(ctiFile)
119
120        camera = selectDevice(h)
121        initializeDevice(camera)
122        print("INIT DONE")
123
124        # configure application node specific features
125        camera.remote_device.node_map.ChunkModeActive.value = True
126        camera.remote_device.node_map.ChunkSelector.value = "Scan3dCoordinateScale"
127        camera.remote_device.node_map.ChunkEnable.value = True
128        camera.remote_device.node_map.ChunkSelector.value = "Scan3dCoordinateOffset"
129        camera.remote_device.node_map.ChunkEnable.value = True
130
131        camera.remote_device.node_map.Scan3dCoordinateSelector.value = "CoordinateC"
132        camera.remote_device.node_map.Scan3dCoordinateScale.value = 0.1
133        camera.remote_device.node_map.Scan3dCoordinateOffset.value = -15000.0
134
135        # setup plot
136        gridsize = (1, 2)
137        fig = plt.figure()
138        axOrgSurf = plt.subplot2grid(gridsize, (0, 0))
139        axRWSurf = plt.subplot2grid(gridsize, (0, 1))
140        fig.suptitle('Measurements')
141        axOrgSurf.set_title('Original Surface')

```

(continues on next page)

```

141     imOrgSurf = axOrgSurf.imshow(np.ones([542, 512]))
142     axRWSurf.set_title('Real-World Surface')
143     imRWSurf = axRWSurf.imshow(np.ones([542, 512]))
144     plt.show(block=False)
145     plt.draw()
146     plt.pause(0.1)
147
148     # acquire data
149     print('start acquisition')
150     camera.start()
151     for itr in range(0, 5):
152         camera.remote_device.node_map.TriggerSelector.value = "FrameStart"
153         camera.remote_device.node_map.TriggerSoftware.execute()
154
155         buffer = camera.fetch(timeout=10.0)
156
157         camera.remote_device.node_map.ChunkScan3dCoordinateSelector.value =
↪ "CoordinateC"
158         coordCScale = camera.remote_device.node_map.ChunkScan3dCoordinateScale.
↪ value
159         coordCOffset = camera.remote_device.node_map.ChunkScan3dCoordinateOffset.
↪ value
160         print(f"ChunkScan3dCoordinateScale: {coordCScale},
↪ ChunkScan3dCoordinateOffset: {coordCOffset}")
161
162         height = buffer.payload.components[0].height
163         width = buffer.payload.components[0].width
164         orgSurf = buffer.payload.components[0].data.reshape(height, width)
165         orgSurf = np.copy(orgSurf)
166         rwSurf = orgSurf * coordCScale + coordCOffset
167         orgSurfMean = np.mean(orgSurf)
168         rwSurfMean = np.mean(rwSurf)
169         print(f"Surface Mean Values: Original {orgSurfMean}, Real-Word:
↪ {rwSurfMean}")
170
171         imOrgSurf.set_data(orgSurf)
172         imOrgSurf.set_clim(vmin=np.min(orgSurf), vmax=np.max(orgSurf))
173         imOrgSurf.set_extent((0, width, height, 0))
174
175         imRWSurf.set_data(rwSurf)
176         imRWSurf.set_clim(vmin=np.min(rwSurf), vmax=np.max(rwSurf))
177         imRWSurf.set_extent((0, width, height, 0))
178
179         plt.draw()
180         plt.pause(0.1)
181
182         buffer.queue()
183     camera.stop()
184 except Exception as error:
185     print(error)
186
187 camera.destroy()
188 print("SCRIPT END")

```

5 Document History

Version	Date	Changes
15.2.0.0	19-05-2022	initial version